

## < Nonlinear model predictive control (NMPC) 8/29 >

linear MPC와의 가장 큰 차이점은 system이 같아 미래 state 표현도 다른 것

Linear:  $x_{k+1} = Ax_k + Bu_k$

Nonlinear:  $x_{k+1} = f(x_k, u_k)$

따라서 nonlinear은  $x_{k+1}$ 을  $u_0 \sim u_k, x_0$ 로 나타내려면

$$x_1 = f(x_0, u_0)$$

$$x_2 = f(x_1, u_1) \rightarrow x_2 = f(f(x_0, u_0), u_1)$$

$$x_3 = f(x_2, u_2) \rightarrow x_3 = f(f(f(x_0, u_0), u_1), u_2)$$

linear system에서 사용하던  $X = Ax_0 + Bu$ 를 사용할 수 없다.

하지만 여전히  $x_1 \sim x_N$ 은  $U$ 의 함수이다.

$$X = \Phi(x_0, U)$$

NMPC의 기본 optimization problem은 다음과 같다.

$$J^*(x_0) = \min_{X, U} \left[ \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k) + x_N^T P x_N \right]$$

such that  $x_0 = x(t)$

$$x_{k+1} = f(x_k, u_k) \quad k=0, \dots, N-1$$

$$x_k \in X$$

$$u_k \in U$$

$$x_N \in X_f$$

★ continuous-time nonlinear dynamics로 discrete dynamics 만들기

①  $x_{k+1} = x_k + (dt) \times f(x_k, u_k)$  ... 부정확,

② Runge-Kutta 4th order method로 dt 동안 적분해  $x_{k+1}$  구함

[정리]

$$J^*(x(t)) = \min_{x, u} \left[ x_N^T P x_N + \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k) \right]$$

such that  $x_0 = x(t)$

$$x_{k+1} = f(x_k, u_k) \quad k=0, 1, \dots, N-1$$

$$x_k \in \mathcal{X}$$

$$u_k \in \mathcal{U}$$

$$x_N \in \mathcal{X}_f$$

↳ NMPC는 각 step에서 위의 식을 해결하는 controller이다.

direct shooting / direct transcription / direct collocation 의 3가지

접근법이 있다.

위의 식을 풀기 위해 SQP를 NLP (Nonlinear Programming) solver로

사용하는데 각 접근법마다 SQP의 적용방식이 다르다.

\*  $U^*$ 를 구하는 전체적인 그림.

하나의 time step  $k=4$ 로 하자.  $k=4$ 에서 MPC는  $N$ 개 step의 미래를 보고

$U_4^*$ 을 정해야 한다.  $U_4^*$ 를 구하기 위해 SQP를 사용한다.  $j$ 개의 iteration에 대해

$$U_4^{(0)} \xrightarrow{\text{SQP}} U_4^{(1)} \xrightarrow{\text{SQP}} U_4^{(2)} \xrightarrow{\text{SQP}} \dots \xrightarrow{\text{SQP}} U_4^{(j)} \xrightarrow{\text{SQP}} U_4^*$$

이런 식의 그림이다. 즉 SQP는  $U^{(j)}$ 에서  $U^{(j+1)}$ 을 어떻게 만들지 결정하는 식이다.

propagate 하면서 확인하는 것은  $j$ 개의 candidate이 constraint를 만족하고 cost가

최소화되었는지도이다. SQP는  $U^{(j)}$ 를 개선해  $U^{(j+1)}$ 을 내놓으므로 cost가 줄어들다가

더 이상 유의미하게 줄어들지 않는다고 판단하면 그 feasible  $U^{(j)}$ 를  $U^*$ 로 정한다.

혹은 propagate 할수록 cost가 줄어들지만 동시에 constraint를 깨는 순간 벗어나버린다면

그전에 끊어 feasible optimal 나를 구한다. 한 step이 진행되는 나만의 시간 동안

iteration을 돌렸는데 여전히 infeasible 일 수도 있다. 또는 cost가 크지 않게

감소하는데 시간 제한에 의해 cost를 충분히 최소화하지 못한 수도 있다. 그러므로

마지막 candidate 인  $U^{(j)}$ 가 항상  $U^*$ 라고 볼 수는 없다

## < Direct Transcription 8/29 >

현재 상태  $x_0 = x(t)$

predicted Horizon :  $H$

nonlinear discrete dynamics :  $x_{k+1} = f(x_k, u_k)$

Direct transcription에서는 optimizer의 decision variable로 다음을 쓴다.

$$z = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \\ u_0 \\ \vdots \\ u_{N-1} \end{bmatrix} = \begin{bmatrix} X \\ U \end{bmatrix}$$

그냥 이경계만 두면 optimizer가  $x$ 를 아무렇게나 고르므로 dynamics constraint를 적용시킨다.

$$x_1 = f(x_0, u_0)$$

$$x_2 = f(x_1, u_1)$$

$$x_3 = f(x_2, u_2)$$

:

$$x_N = f(x_{N-1}, u_{N-1})$$

$$\rightarrow C_k(x_k, u_k, x_{k+1}) = x_{k+1} - f(x_k, u_k) = 0$$

optimizer는 state를 자유롭게 고를 수 있지만  $x_{k+1} = f(x_k, u_k)$ 를 만족시키는 state만 최종 solution으로 인정한다.

\* tracking error의 cost 정의

$$J(z) = \sum_{k=0}^{N-1} [(x_k - x_k^r)^T Q (x_k - x_k^r) + (u_k - u_k^r)^T R (u_k - u_k^r)] \\ + (x_N - x_N^r)^T P (x_N - x_N^r)$$

$\min_z J(z)$  를 만족시키는  $z$  를 구하는 것이 목표임.

\* 우리가 풀어야하는 NLP 정의

$$\min_z J(z)$$

$$\text{such that } c_k(z) = x_{k+1} - f(x_k, u_k) = 0$$

$$g(z) \leq 0$$

해당 Nonlinear Program을 풀기 위해 solver로 SQP 사용

\* 처음에 사용할 initial  $z$  정하기

$v = [0, 0, \dots, 0]$  이라 하면  $x_0$ 과  $u_0, \dots, u_{N-1}$ 로 시작할 수 있음. 이렇게 얻은  $x_1, x_2, \dots$

$x_N$ 을  $x^{(0)}$ 라 하자.

$$\text{그러면 } z^{(0)} = \begin{bmatrix} x^{(0)} \\ v^{(0)} \end{bmatrix}$$

SQP iteration  $j$ 의 현재 candidate는  $z^{(j)}$ 라 하면 이제

$$z^{(j+1)} = z^{(j)} + \Delta z$$

에서  $\Delta z$ 를 구해야 한다. 이를 위해 Taylor expansion

\* cost 변형 ( $\Delta z$ 에 대한 quadratic form으로)

$$z = z^{(j)} + \Delta z \text{ 라 하고 } J(z) = J(z^{(j)} + \Delta z) \text{ 를 전개}$$

$$J(z^{(j)} + \Delta z) \approx J(z^{(j)}) + \nabla J(z^{(j)})^T \Delta z + \frac{1}{2} \Delta z^T H_j \Delta z$$

Hessian or

Lagrangian Hessian approximation

$$\text{223} \quad \min \left[ J(z^{(j)}) + \nabla J(z^{(j)})^T \Delta z + \frac{1}{2} \Delta z^T H_j \Delta z \right] \text{ 인데 } z^{(j)} \text{ 는 } \Delta z \text{ 와 무관하므로}$$

$$\rightarrow \min_{\Delta z} \left[ \frac{1}{2} \Delta z^T H_j \Delta z + \nabla J(z^{(j)})^T \Delta z \right]$$

## \* Nonlinear dynamics의 linearization

dynamics constraint는  $C_k = x_{k+1} - f(x_k, u_k) = 0$

$i$ th iteration  $x_k^{(i)}, u_k^{(i)}$ 를  $\bar{x}_k, \bar{u}_k$ 라 하면

그냥  $i$ 번째 candidate이 가져온  $x, u$ 를  
잠시  $\bar{x}_k, \bar{u}_k$ 로 표현하는 것이다.

다음 iteration candidate의  $x_k, u_k$ 는

$$x_k = \bar{x}_k + \delta x_k$$

$$u_k = \bar{u}_k + \delta u_k$$

이것로 nonlinear dynamics를 만족하므로 다음 만족 후 Taylor expansion

$$\bar{x}_{k+1} + \delta x_{k+1} = f(\bar{x}_k + \delta x_k, \bar{u}_k + \delta u_k)$$

$$\approx f(\bar{x}_k, \bar{u}_k) + A_k \delta x_k + B_k \delta u_k$$

$$\left. \frac{\partial f}{\partial x} \right|_{\bar{x}_k, \bar{u}_k}$$

$$\left. \frac{\partial f}{\partial u} \right|_{\bar{x}_k, \bar{u}_k}$$

$$\text{따라서 } \delta x_{k+1} - A_k \delta x_k - B_k \delta u_k = -[\bar{x}_{k+1} - f(\bar{x}_k, \bar{u}_k)]$$

$$\delta x_{k+1} - A_k \delta x_k - B_k \delta u_k = -dx_k$$

현재 trajectory  $\bar{x}_k, \bar{u}_k$ 의  
dynamics violation

\* Actuator constraint을 correction으로 표현

현재 candidate은  $\bar{u}_k$

새로운 control은  $u_k = \bar{u}_k + \delta u_k$

기존의 constraint이  $-u_{\max} \leq u_k \leq u_{\max}$  이므로

$$-u_{\max} \leq \bar{u}_k + \delta u_k \leq u_{\max}$$

$$-u_{\max} - \bar{u}_k \leq \delta u_k \leq u_{\max} - \bar{u}_k$$

\* State constraint을 correction으로 표현

$$x_k \leq x_{\max}$$

new  $x$ 는  $x_k = \bar{x}_k + \delta x_k$

$$\bar{x}_k + \delta x_k \leq x_{\max}$$

$$\delta x_k \leq x_{\max} - \bar{x}_k$$

linear constraint일 때

state constraint이  $g(x_k) \leq 0$  이면

Taylor expansion 해서 최종 결과가

$$g(\bar{x}_k) + \nabla g(\bar{x}_k)^T \delta x_k \leq 0$$

nonlinear constraint일 때

# \* 하나의 Iteration의 QP

결정변수는  $\Delta z$

$$\Delta z = \begin{bmatrix} \delta x_1 \\ \vdots \\ \delta x_n \\ \delta u_0 \\ \vdots \\ \delta u_{N-1} \end{bmatrix}$$

$n(N-1)$   
 $n(N-1) \sim 2n(N-1)$   
 $n(N-1) \sim n(N-1)$   
 $k=1$

sQP iteration j에서 푸는 문제는

$\nabla J(z^{(j)})$

$$\min_{\Delta z} \left[ \frac{1}{2} \Delta z^T H_j \Delta z + \nabla J(z^{(j)})^T \Delta z \right]$$

$$\text{such that } \delta x_{k+1} - A_k \delta x_k - B_k \delta u_k = -d_k$$

linearized state constraint

linearized u constraint

↳ 간단한 Quadratic Program

현재 trajectory 주어진에서 QP<sup>(0)</sup> QP<sup>(1)</sup> QP<sup>(2)</sup> 반복해서 해결

$$\text{QP solution: } \Delta z^{*(j)} = \begin{bmatrix} \delta x^{*(j)} \\ \delta u^{*(j)} \end{bmatrix} \quad \text{로 주어진대로 update는}$$

$$\begin{cases} v^{(j+1)} = v^{(j)} + \alpha_j \delta v^{*(j)} \\ x^{(j+1)} = x^{(j)} + \alpha_j \delta x^{*(j)} \end{cases}$$

\* optimal  $V$  판단 시점

$\| \Delta z^{(i)} \|$  이 충분히 작고 ①

$\| C(z^{(i)}) \|$  이 충분히 작고 ②

KKT optimality residual도 충분히 작으면 ③

$$(x^{(i)}, v^{(i)}) \approx (x^*, v^*)$$

최종 solution :  $v_t^* = \begin{bmatrix} u_{t+t}^* \\ u_{t+t+1}^* \\ \vdots \\ u_{t+N-1+t}^* \end{bmatrix}$

→ 실제 사용은 첫번째만

다음 step의 initial guess로 사용 (for warm start)

$$v_{t+1}^{(0)} = \begin{bmatrix} u_{t+t+1}^* \\ \vdots \\ u_{t+N-1+t}^* \\ u_{t+t+1}^* \end{bmatrix}$$

$$x_{t+1}^{(0)} = \begin{bmatrix} f(x_0, u_{t+t+1}^*) \\ f(x_{1|t+1}^{(0)}, u_{t+t+2}^*) \\ \vdots \\ f(x_{N-1|t+1}^{(0)}, u_{t+t+1}^*) \end{bmatrix}$$

여로 코드

[참고]

① 한번에 optimal control  $U^*$ 로 이동하지 못하고  $j$ 번의 iteration이 필요한 이유

$z^{(0)}$ 에서  $z^*$ 까지의 correction을 바로 아는 것은 불가능하다. SQP는 매순간

local problem만 보고  $\Delta z^{(0)}$ 을 구한다. 그 다음은  $\Delta z^{(1)}$  ...

결과적으로  $z^* \approx z^{(0)} + \alpha_0 \Delta z^{(0)} + \alpha_1 \Delta z^{(1)} + \dots$  이다.

이때 SQP가 사용하는 Taylor expansion이 candidate local에서만 성립하기 때문에 신뢰할 수 있는 neighborhood 범위에서 한 step 이동해 nonlinear model을 선형화한다.

이러함이 ' $j$ 번의 iteration'인 것이다.

②  $U_t^*$ 의  $U_{t+1}^*$ 를 제외한 나머지 부분은  $t+1$ 의 initial guess로 이용하는 이유

실제 시스템이 예상과 크게 다르게 움직이지 않았다면

$$U_{t+5}^* \approx U_{t+4}^*$$

$$U_{t+6}^* \approx U_{t+5}^*$$

⋮

일 가능성이 높다. 반면  $U^{(0)} = 0$  이런 식으로 가이드라인 없이 시작하면 optimum까지의

거리가 멀어 비효율적이다. 따라서 전  $t$ 의 나머지  $U_t^*$ 를 가져다가 nonlinear dynamics의

당여  $x_{t+1}^{(0)}$ 을 만들고,  $(x_{t+1}^{(0)}, U_{t+1}^{(0)})$ 로 다음 step SQP를 시작한다.

이러면 dynamics도 만족하는 warm start다.

\* QP 풀기

$$\min_{\Delta z} \left[ \frac{1}{2} \Delta z^T H; \Delta z + \nabla J; \Delta z \right]$$

$$\delta x_{k+1} - A_k \delta x_k - B_k \delta u_k = -d_k \quad : \text{dynamics constraint}$$

$$-U_{\max} - u_k \leq \delta u_k \leq U_{\max} - u_k \quad : \text{actuator constraint}$$

$$g(\bar{x}_k) + \nabla g(\bar{x}_k)^T \delta x_k \leq 0 \quad : \text{state constraint}$$

\* dynamics constraint

$$\begin{array}{c} \begin{matrix} \xrightarrow{N \times N} & \xrightarrow{N \times M} \end{matrix} \\ \left[ \begin{array}{cccc|cccc} I & 0 & \dots & 0 & -B_0 & 0 & 0 & 0 \\ -A_1 I & \dots & 0 & 0 & -B_1 & 0 & \dots & 0 \\ 0 & -A_2 I & \dots & 0 & 0 & 0 & -B_2 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \dots & -A_{N-1} I & 0 & 0 & 0 & -B_{N-1} \end{array} \right] \begin{bmatrix} \delta x_1 \\ \vdots \\ \delta x_N \\ \delta u_0 \\ \vdots \\ \delta u_{N-1} \end{bmatrix} = \begin{bmatrix} -d_0 \\ -d_1 \\ \vdots \\ -d_{N-1} \end{bmatrix} \\ \underbrace{\hspace{1.5cm}}_{N} \quad \underbrace{\hspace{1.5cm}}_{N} \quad \quad \quad 2N \times 3 \quad \quad \quad N \times 3 \end{array}$$

$N \times 2N$